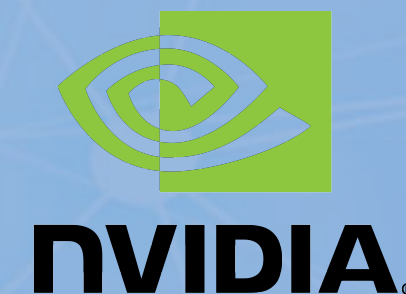


Deep Learning Recommendation System cross-GPU Communication Optimization

Yuxin Chen
Aydin Buluc
Katherine Yelick
John Owens

(UC Davis, NVIDIA)
(LBNL)
(UC Berkeley)
(UC Davis)



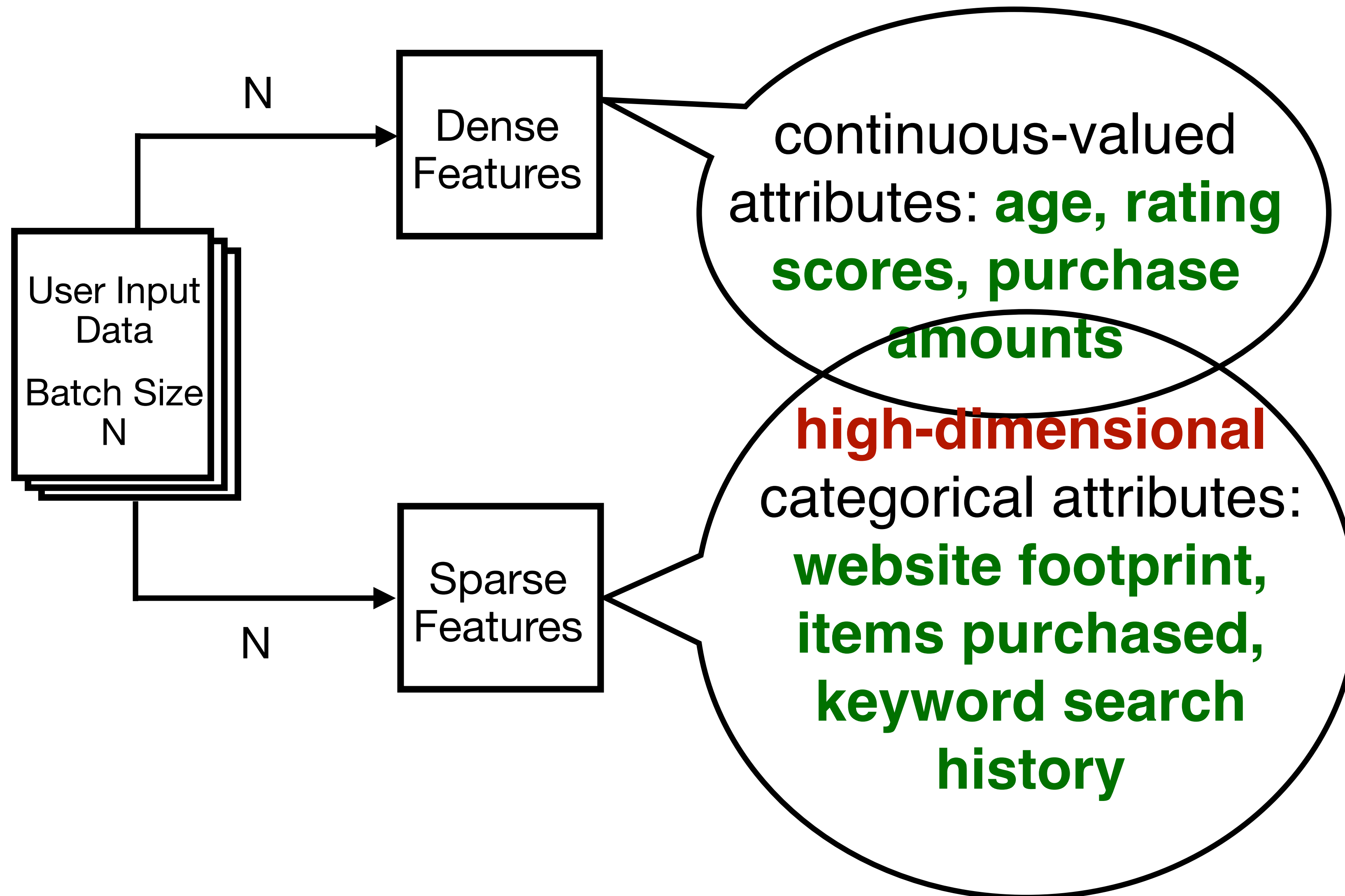
UCDAVIS

Berkeley
UNIVERSITY OF CALIFORNIA

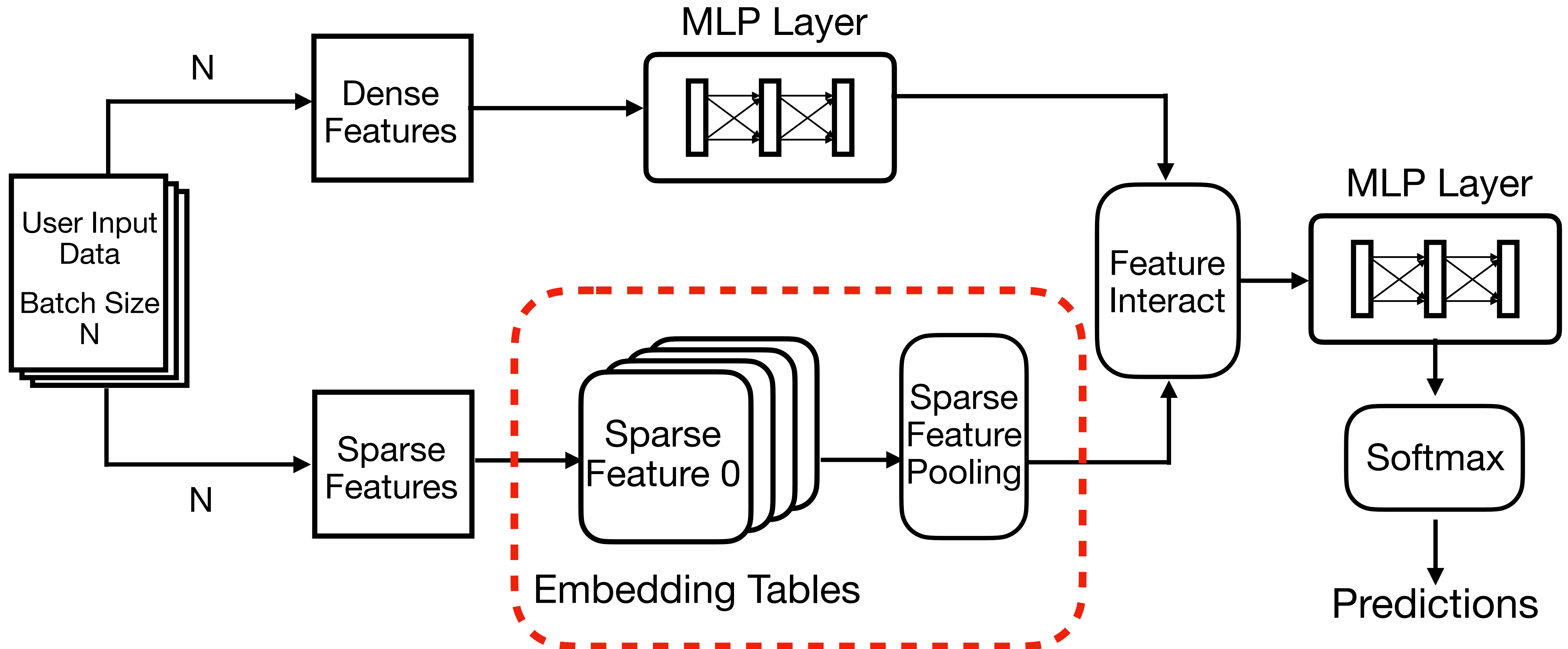
Agenda

- 1 Deep Learning Recommendation Model (DLRM)** 
- 2 DLRM Communication Pattern**
- 3 PGAS Async One-sided Small Messages**
- 4 Case Studies**

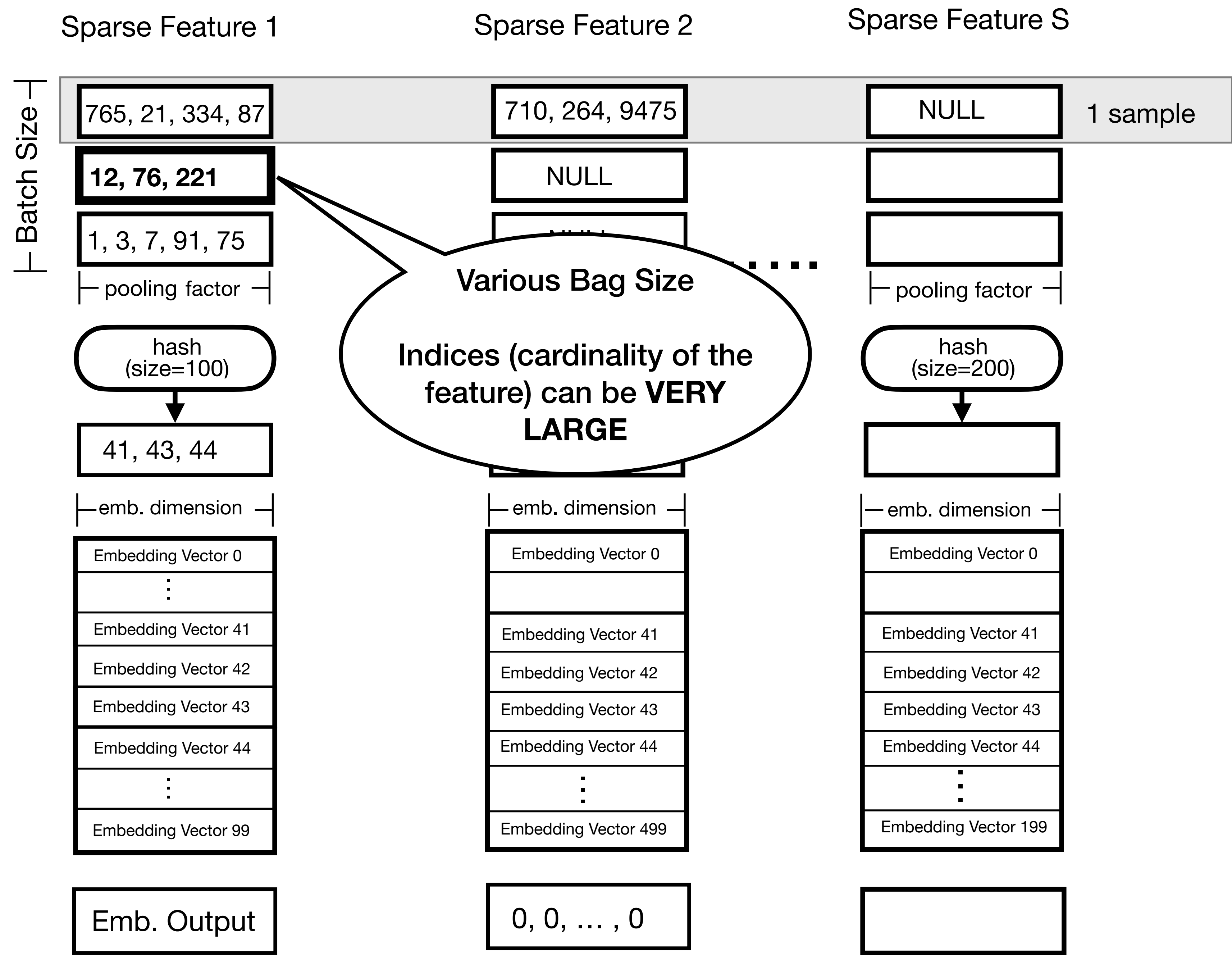
Deep Learning Recommendation Model (DLRM) Inferencing



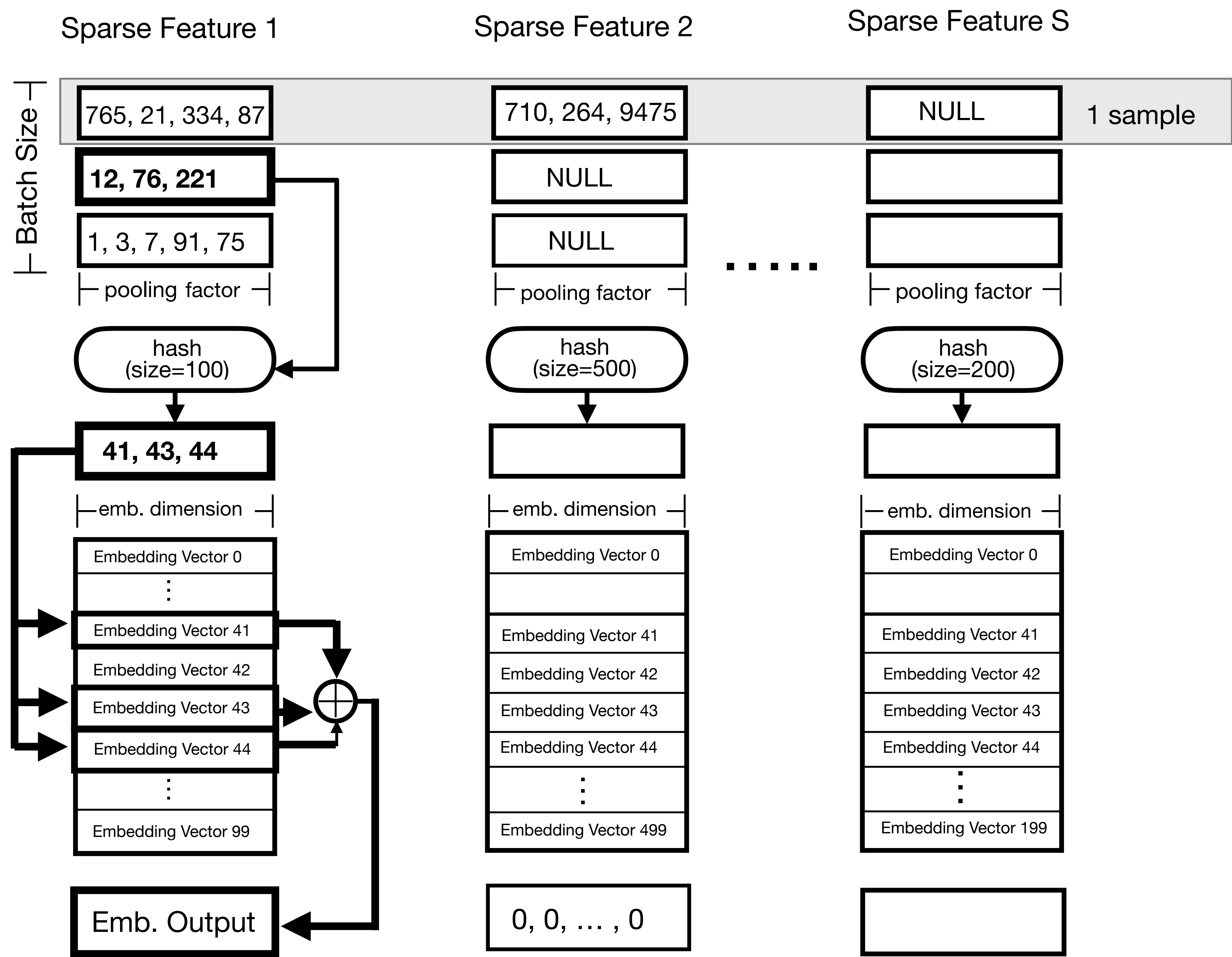
Deep Learning Recommendation Model (DLRM) Inferencing



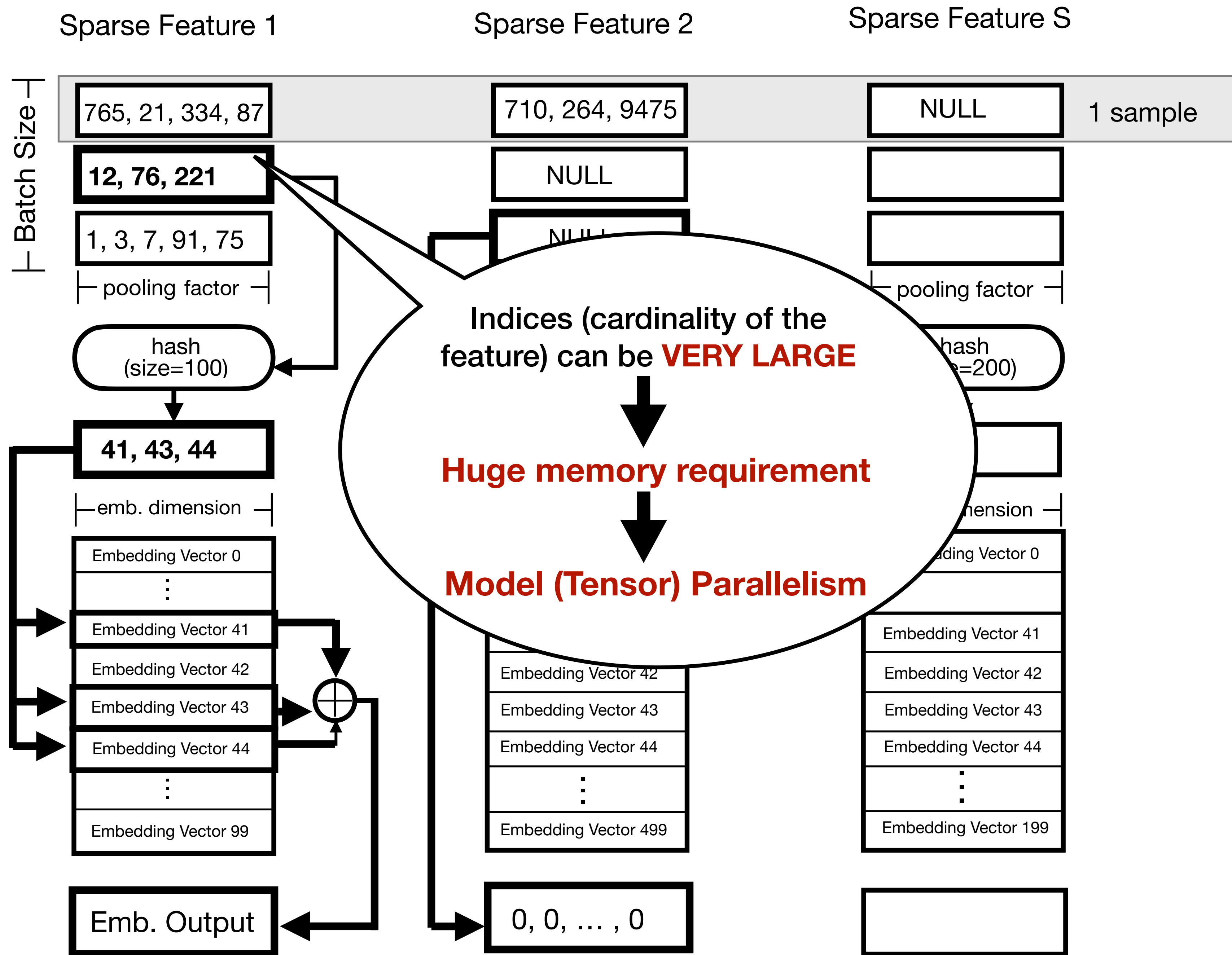
Deep Learning Recommendation Model (DLRM) Inferencing



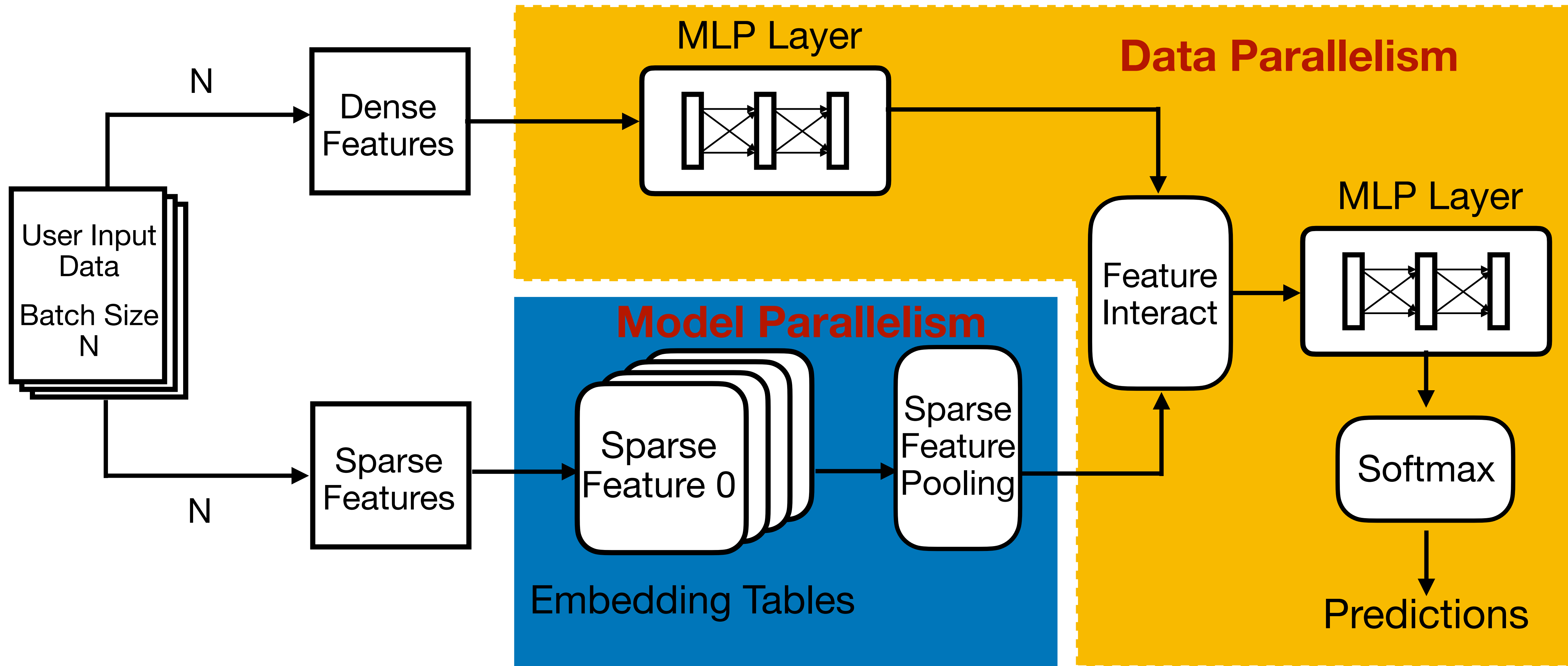
Deep Learning Recommendation Model (DLRM) Inferencing



Deep Learning Recommendation Model (DLRM) Inferencing



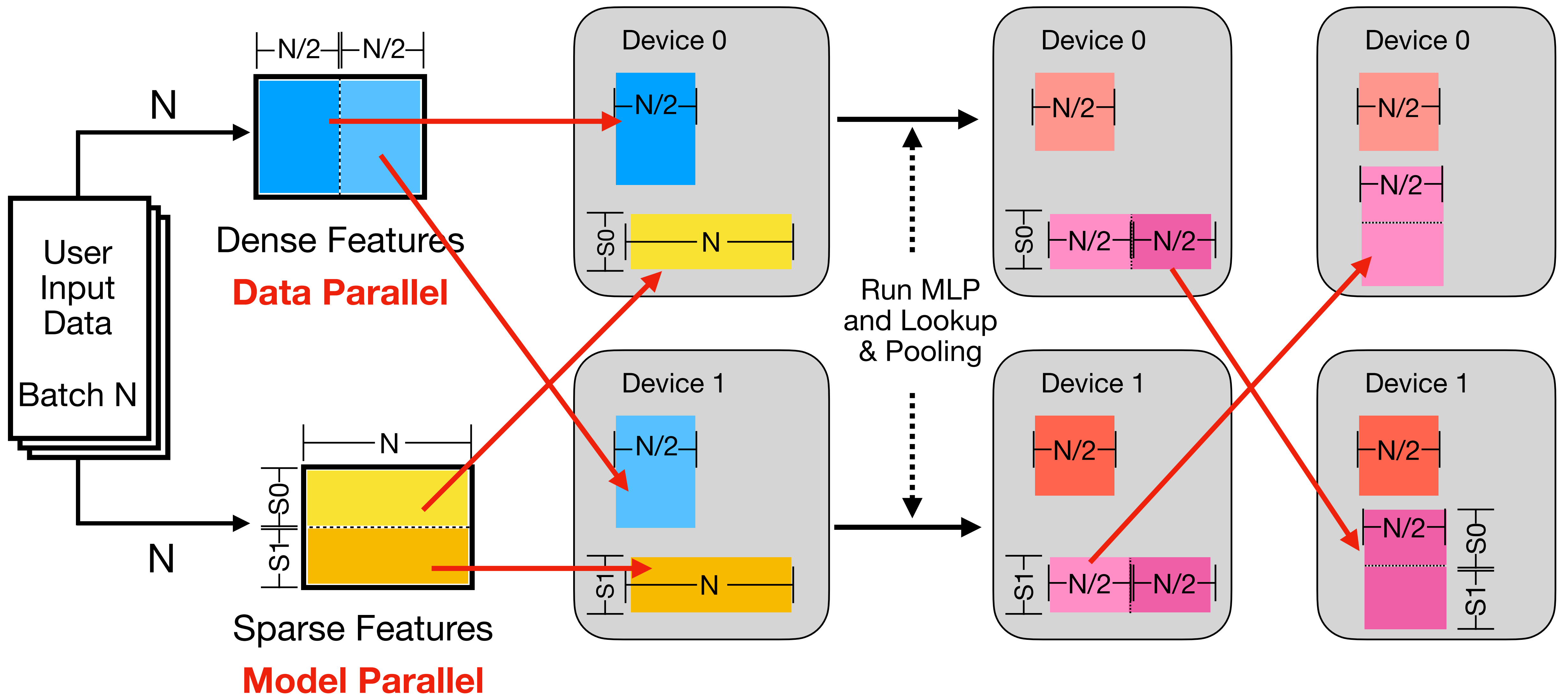
Deep Learning Recommendation Model (DLRM) Inferencing



Agenda

- 1 Deep Learning Recommendation Model (DLRM)**
- 2 DLRM Communication Pattern** 
- 3 PGAS Async One-sided Small Messages**
- 4 Case Studies**

Deep Learning Recommendation Model (DLMR) Inferencing



Deep Learning Recommendation Model (DLMR) Inferencing

Traditional Code

```
sparse_embeddings = apply_emb(sparse_input)
```

```
a2a_req = ext_dist.alltoall(sparse_embeddings)
```

```
dense_embeddings = apply_mlp(dense_input)
```

```
recv = a2a_req.wait()
```

```
sparse_embeddings = unpack(recv)
```

Computation

**Communication
Collective Call**

Unpacking

Agenda

- 1 Deep Learning Recommendation Model (DLRM)**
- 2 DLRM Communication Pattern**
- 3 PGAS Async One-sided Small Messages** 
- 4 Case Studies**

PGAS One-sided Small Asynchronous Messages

Communication Schemes:

- Small

- One-sided Async

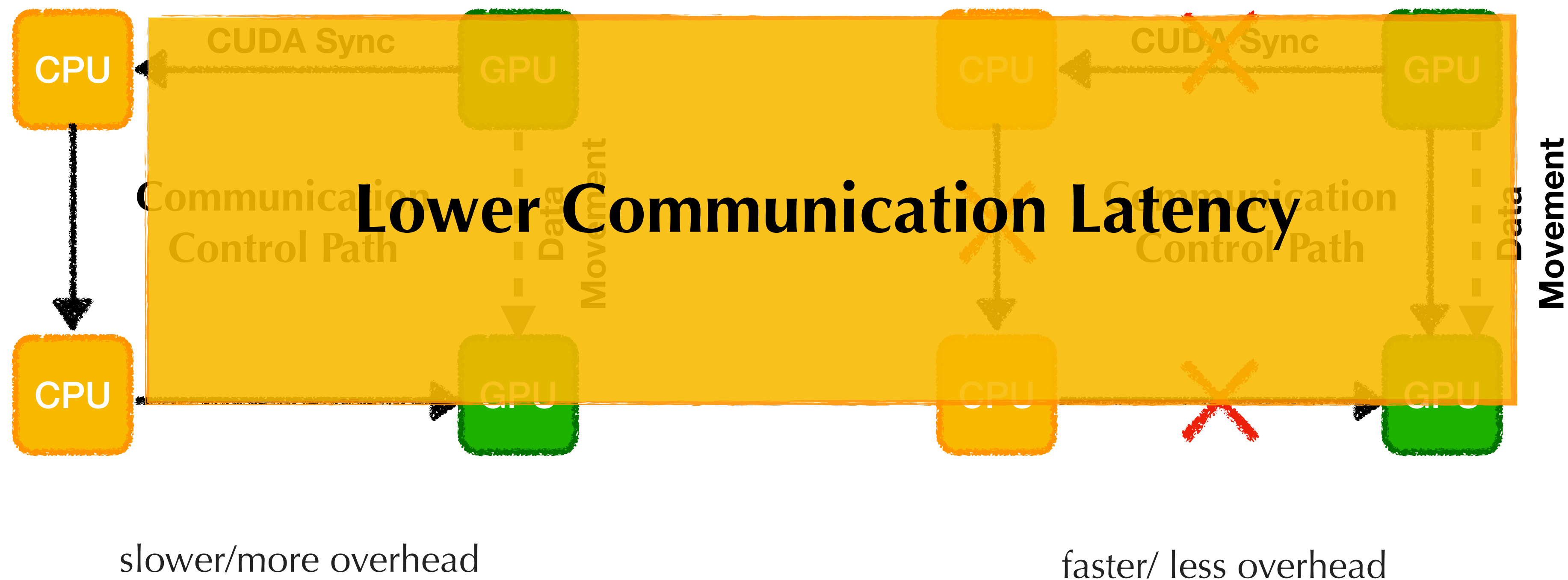
- Inside GPU kernels

What is **BENEFIT?**

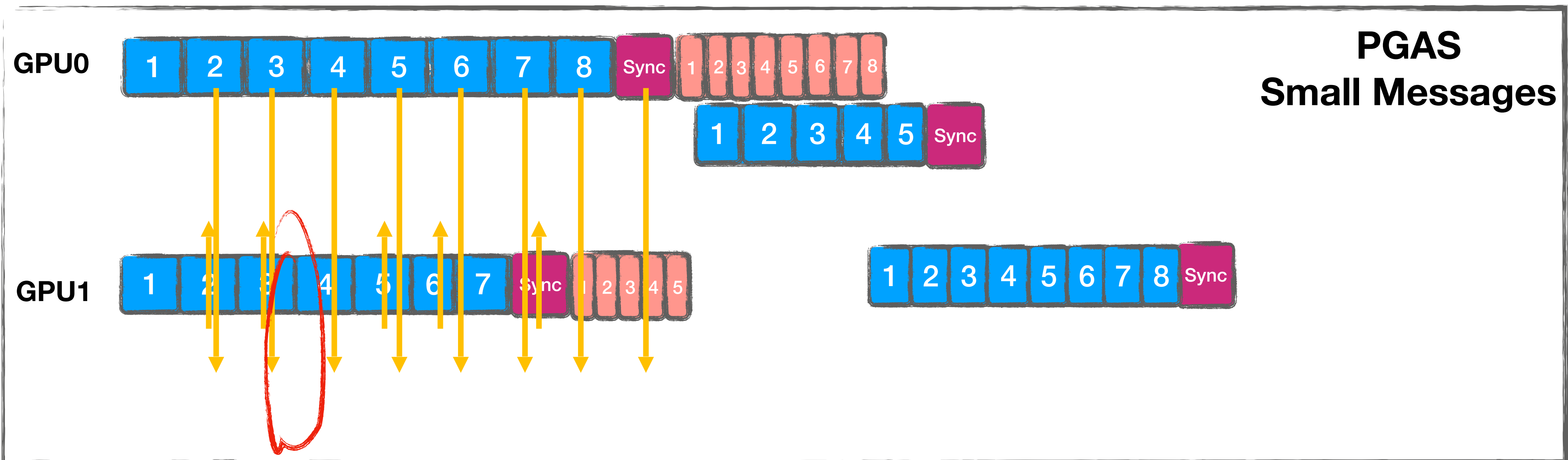
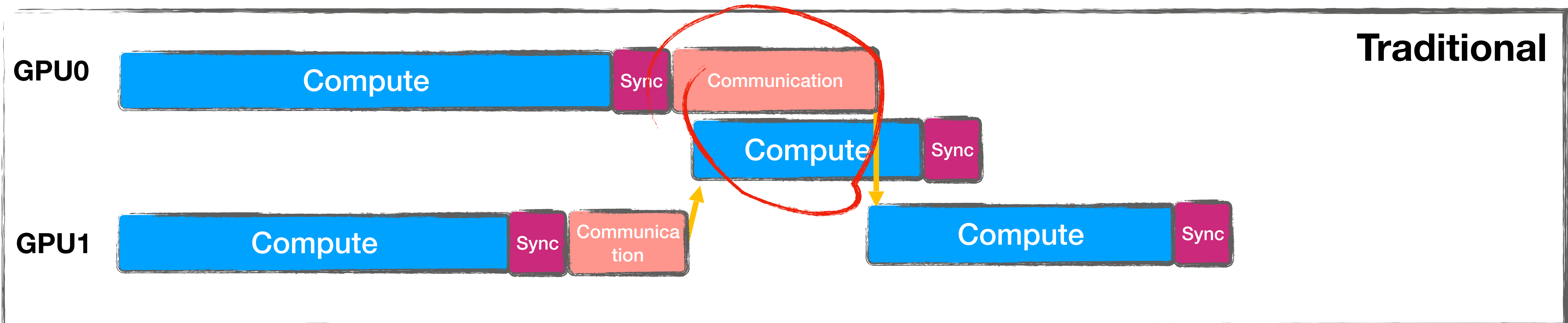
- Remove Send Aggregation & Receive Unpacking,
Writing to Targeting Memory Location Directly

PGAS One-sided Small Asynchronous Messages

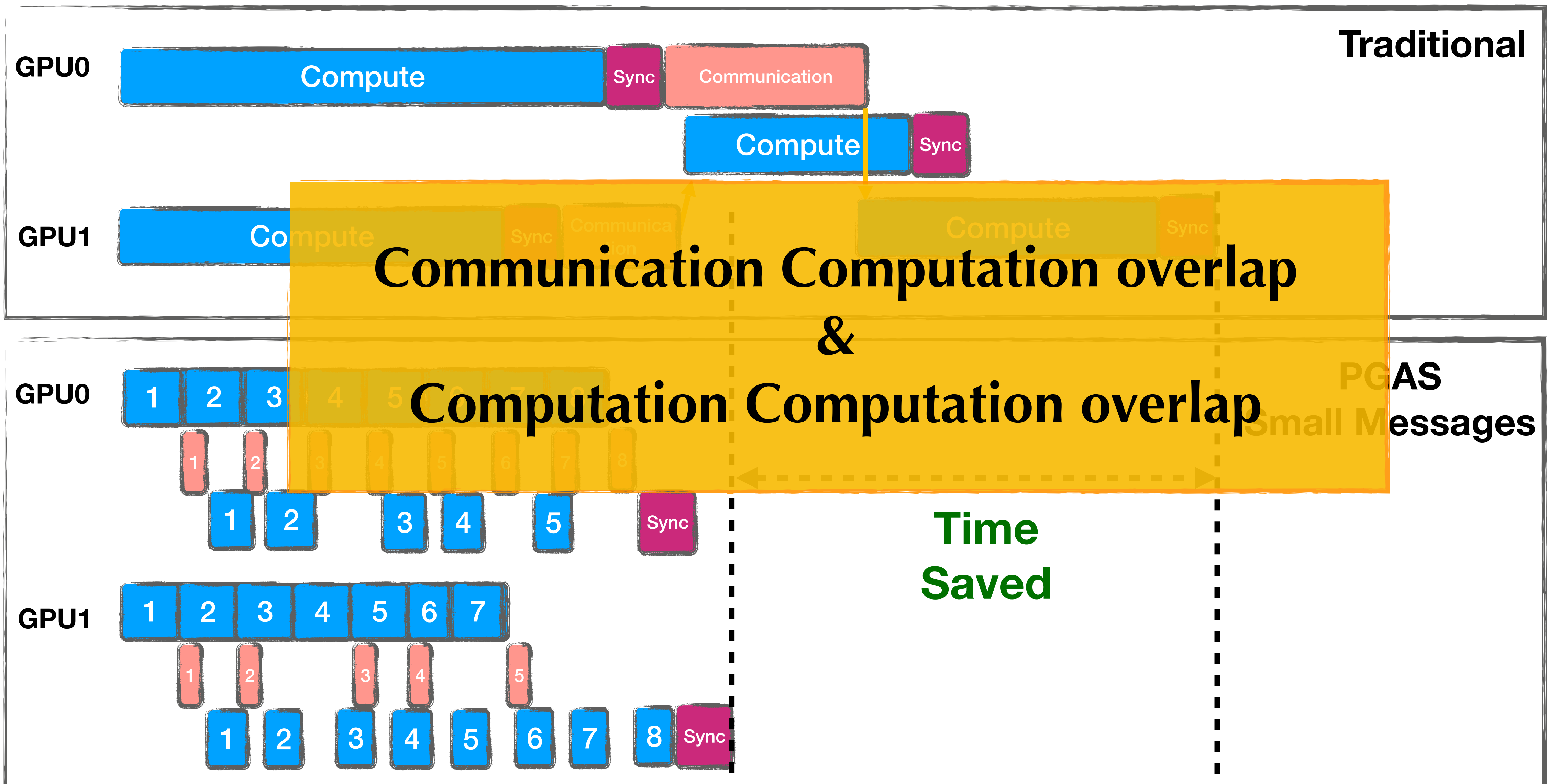
At the Time of Communication.....



PGAS One-sided Small Asynchronous Messages

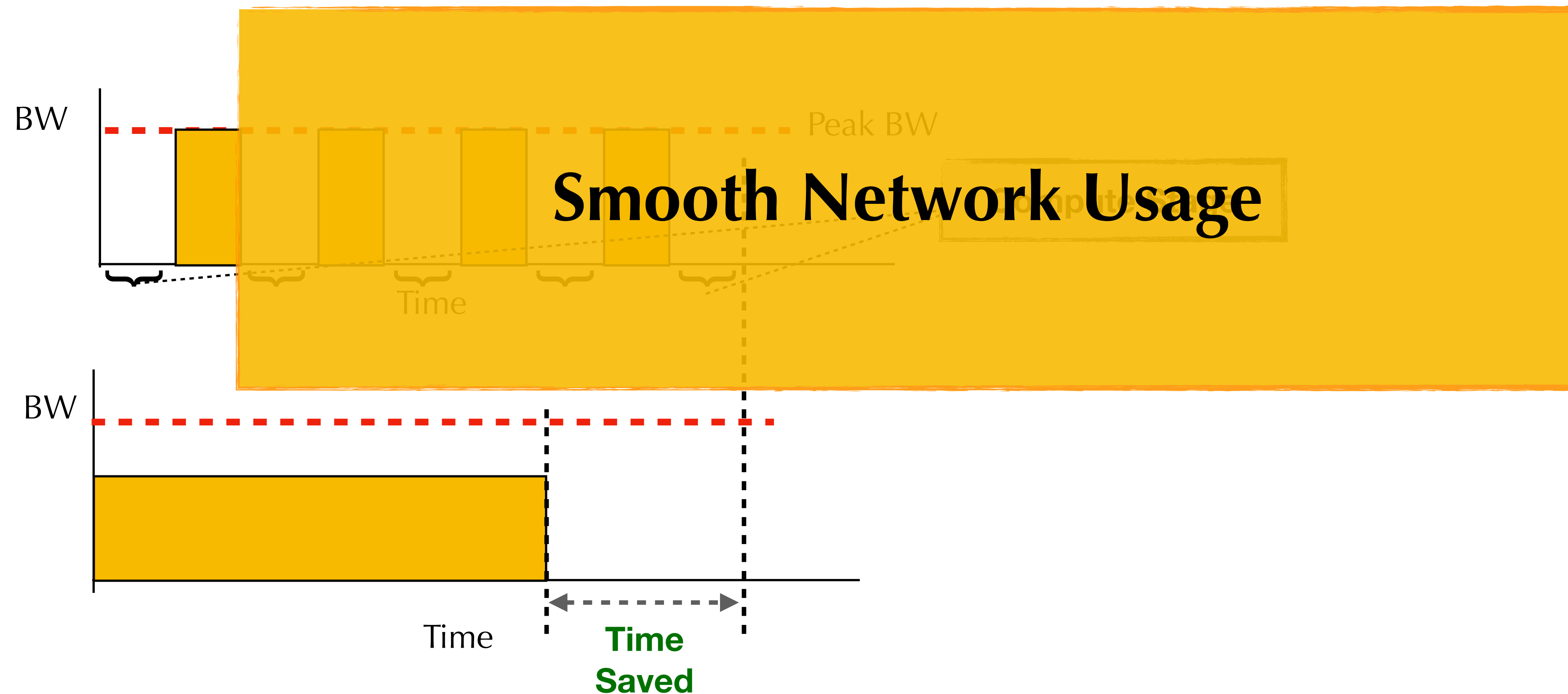


PGAS One-sided Small Asynchronous Messages



PGAS One-sided Small Asynchronous Messages

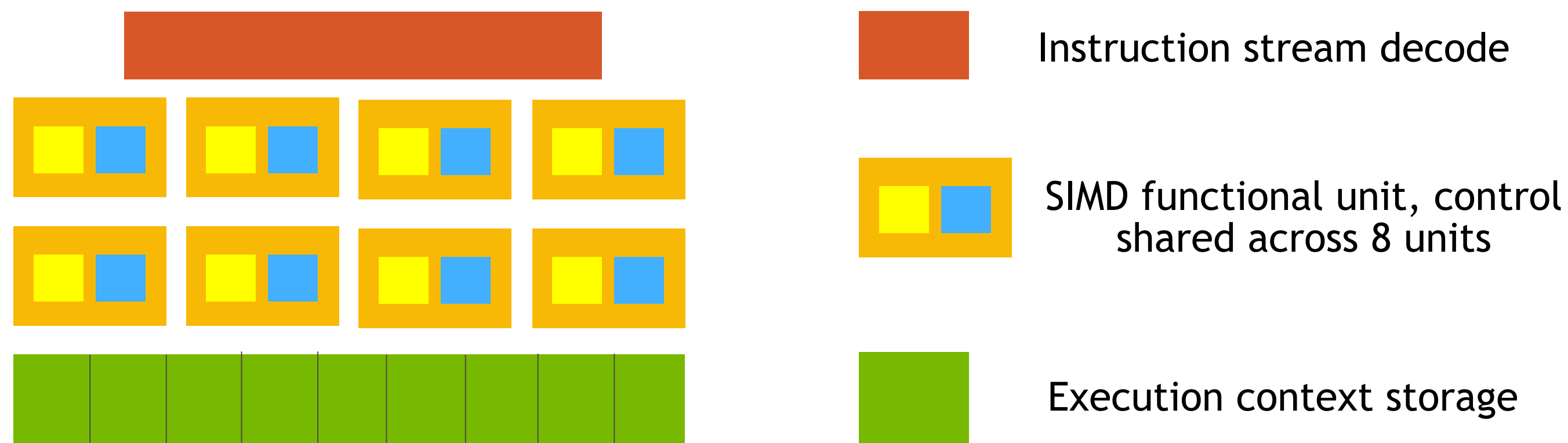
Smooth out the network usage by spreading communication along execution time.



PGAS One-sided Small Asynchronous Messages

Muti-threading requires enough work to hide latency, Do we have enough work?

- Communication size is small thus easy to hide
- GPU uses same philosophy to hide stall



PGAS One-sided Small Asynchronous Messages

When to use PGAS one-sided small asynchronous messages?

- Computation vs Communication ratio

- $> 1:1$ ratio 
 - 1:2 ratio 
 - 1: 100 ratio 

- Hardware used

- NVLink 
 - IB band 
 - Ethernet 

- Memory Access Pattern, TLB size, etc.

PGAS One-sided Small Asynchronous Messages

Traditional Code

```
sparse_embeddings = apply_emb(sparse_input)
```

```
a2a_req = ext_dist.alltoall(sparse_embeddings)
```

```
dense_embeddings = apply_mlp(dense_input)
```

```
recv = a2a_req.wait()
```

```
sparse_embeddings = unpack(recv)
```

Computation

**Communication
Collective Call**

PGAS Code

```
sparse_embeddings = apply_emb_pgas(sparse_input)
```

```
dense_embeddings = apply_mlp(dense_input)
```

**Fine-grained
Interleaved
Communication
& Computation**

PGAS One-sided Small Asynchronous Messages

```
template<typename scalar_t, typename AccuFunc, typename ShardFunc>
__global__ void PGAS_EMB_forward_kernel(
    const PackedTensorAccessor64<scalar_t, 2,
                                   RestrictPtrTraits> weights,
    const PackedTensorAccessor64<int64_t, 1,
                                   RestrictPtrTraits> offsets,
    const PackedTensorAccessor64<int64_t, 1,
                                   RestrictPtrTraits> indices,
    ....
    ShardFunc GetEmbOwnerId) {
    // Compute the embedding lookup and pooling given sparse
    // input offsets and indices and store the embedding result
    // in Vec4T sum
    .....
    auto [pe, output_idx] = GetEmbOwnerId(
        threadIdx.y, blockIdx.x, BatchSize, n_gpus);
    if(pe == gpu_id) {
        // Writing the embeddings locally
        sum.store(outputs[output_idx]);
    }
    else {
        // PGAS writing the embeddings to remote GPU memory
        sum.store(outputs[output_idx], pe);
    }
}
```

Agenda

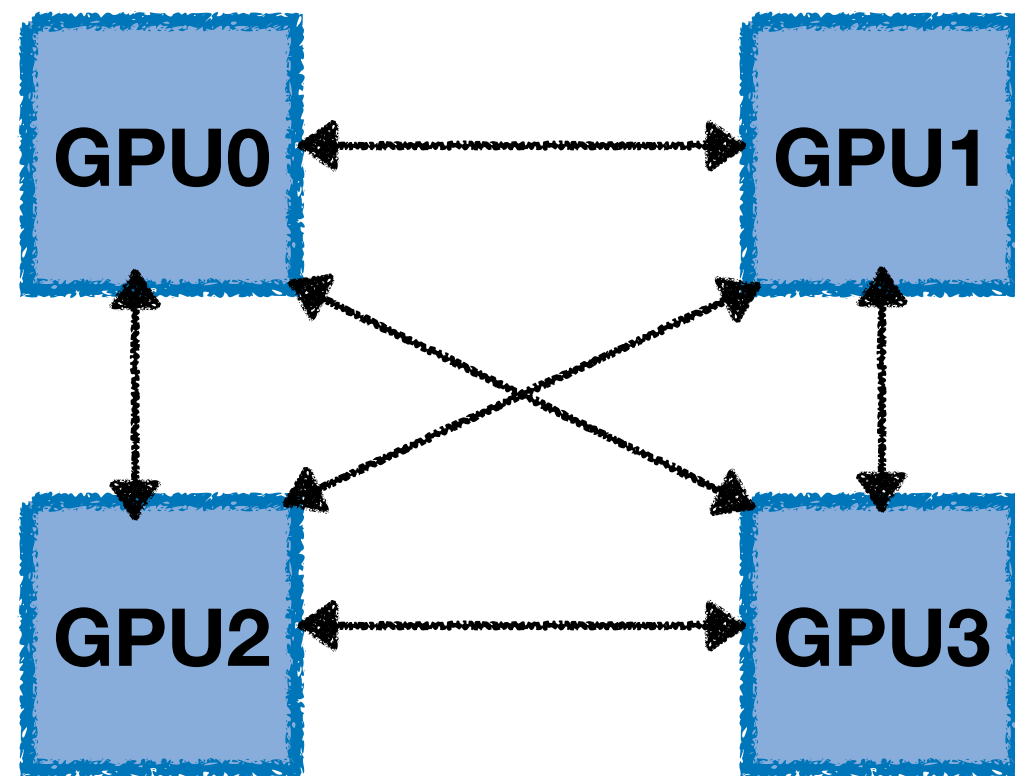
- 1 Deep Learning Recommendation Model (DLRM)**
- 2 DLRM Communication Pattern**
- 3 PGAS Async One-sided Small Messages**
- 4 Case Studies** 

PGAS DLMR Case Studies

Case Studies:

- Computation vs (Communication+Unpack): 1/1
- Baseline: Pytorch+NCCL
- Our approach: PGAS fused implementation

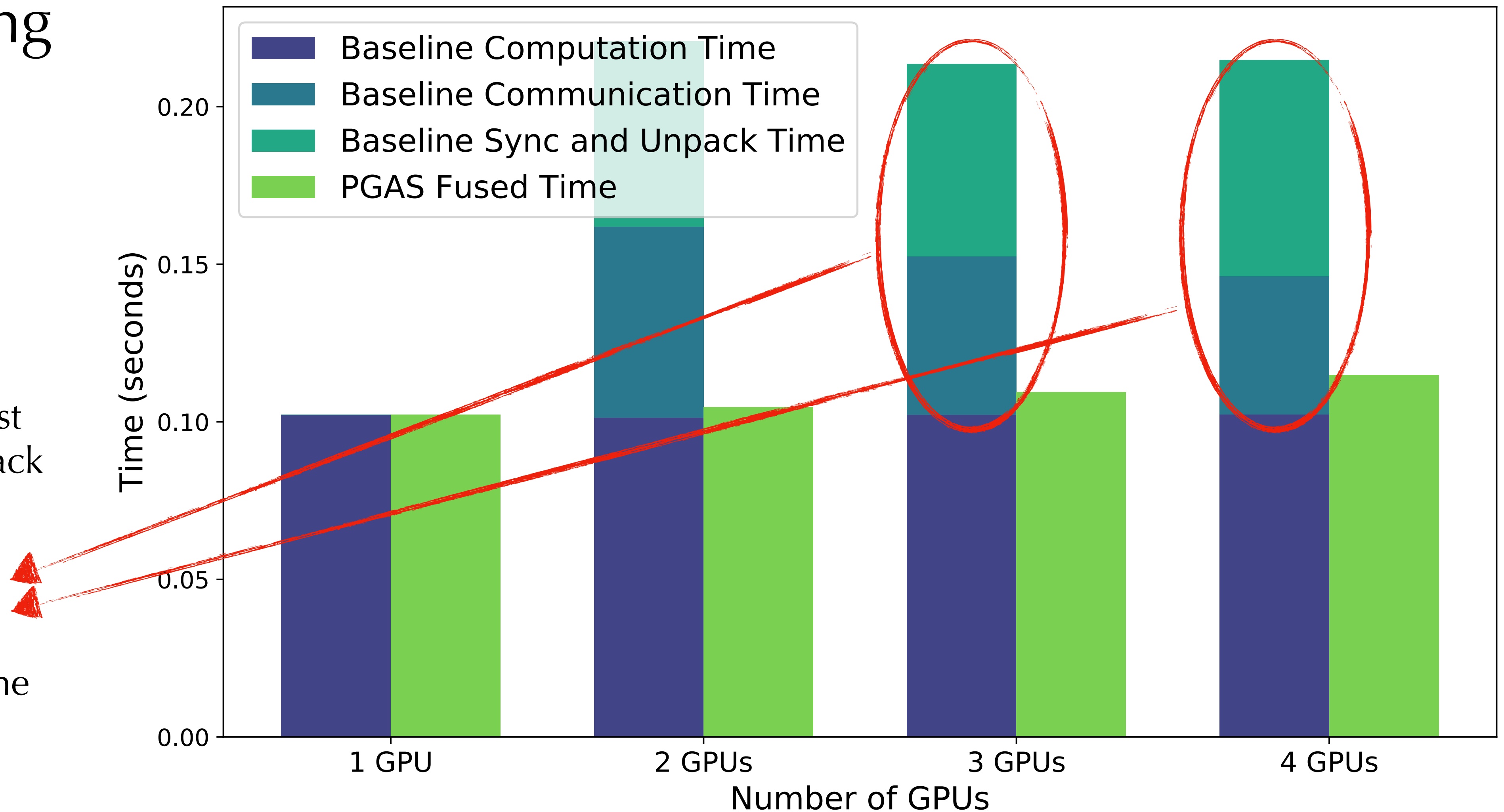
System: NVLink System



PGAS DLMR Weak Scaling

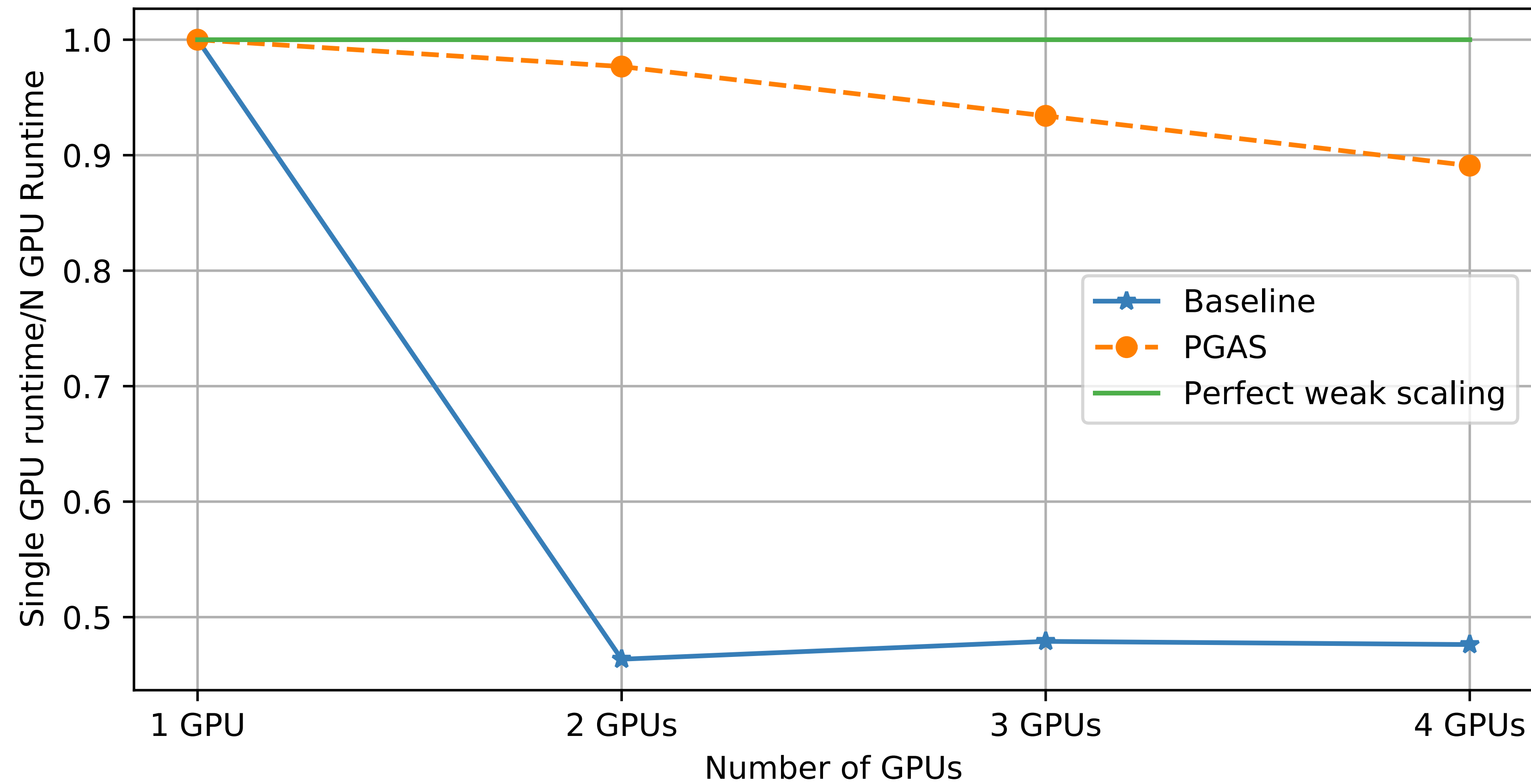
Weak Scaling

- Best overall: **PGAS** implementation
- **PGAS** reduces sync cost and eliminate the unpack operation
- **PGAS** is able to completely hide the communication with the computation



PGAS DLMR Weak Scaling

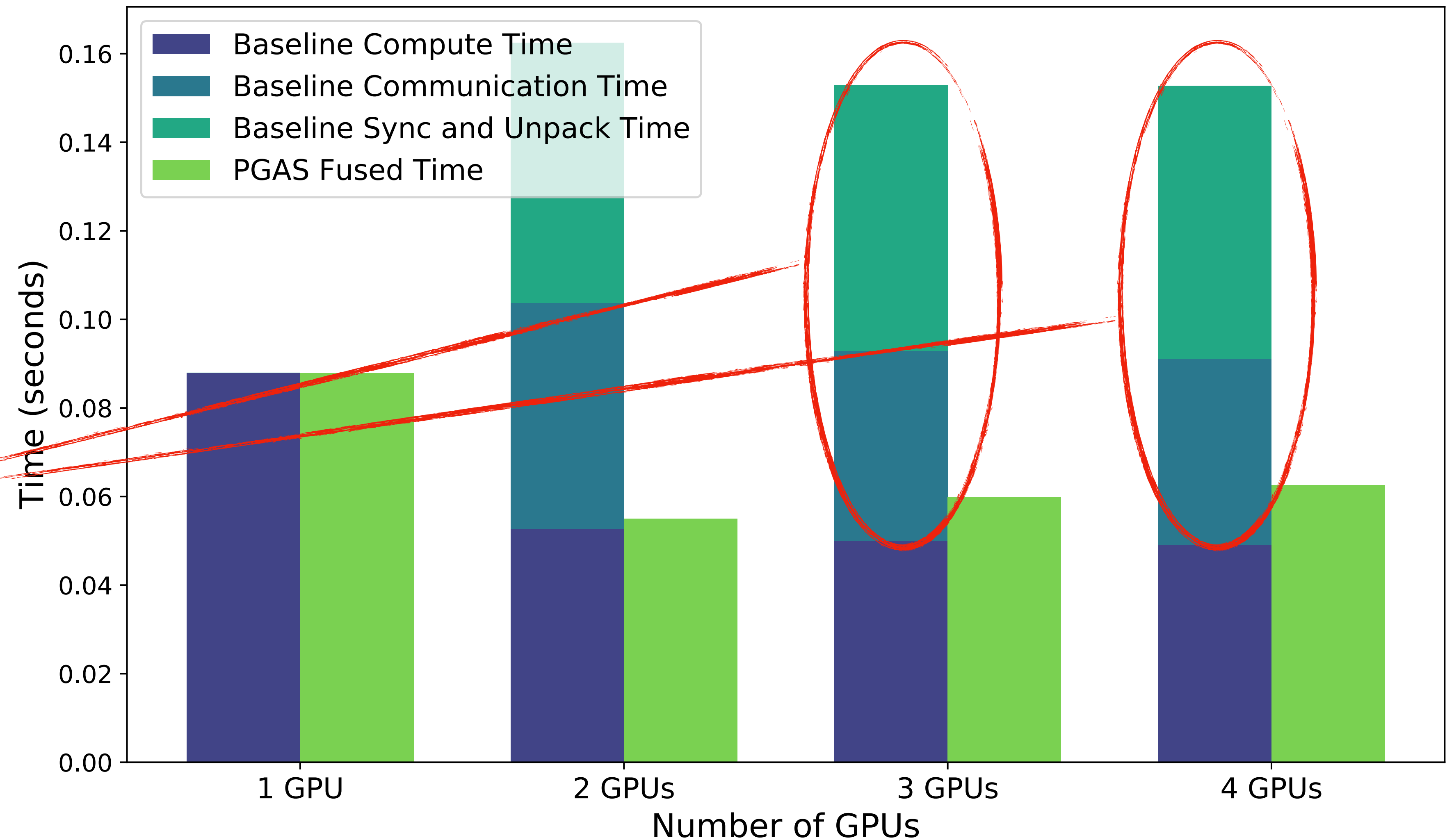
Weak Scaling



PGAS DLMR Strong Scaling

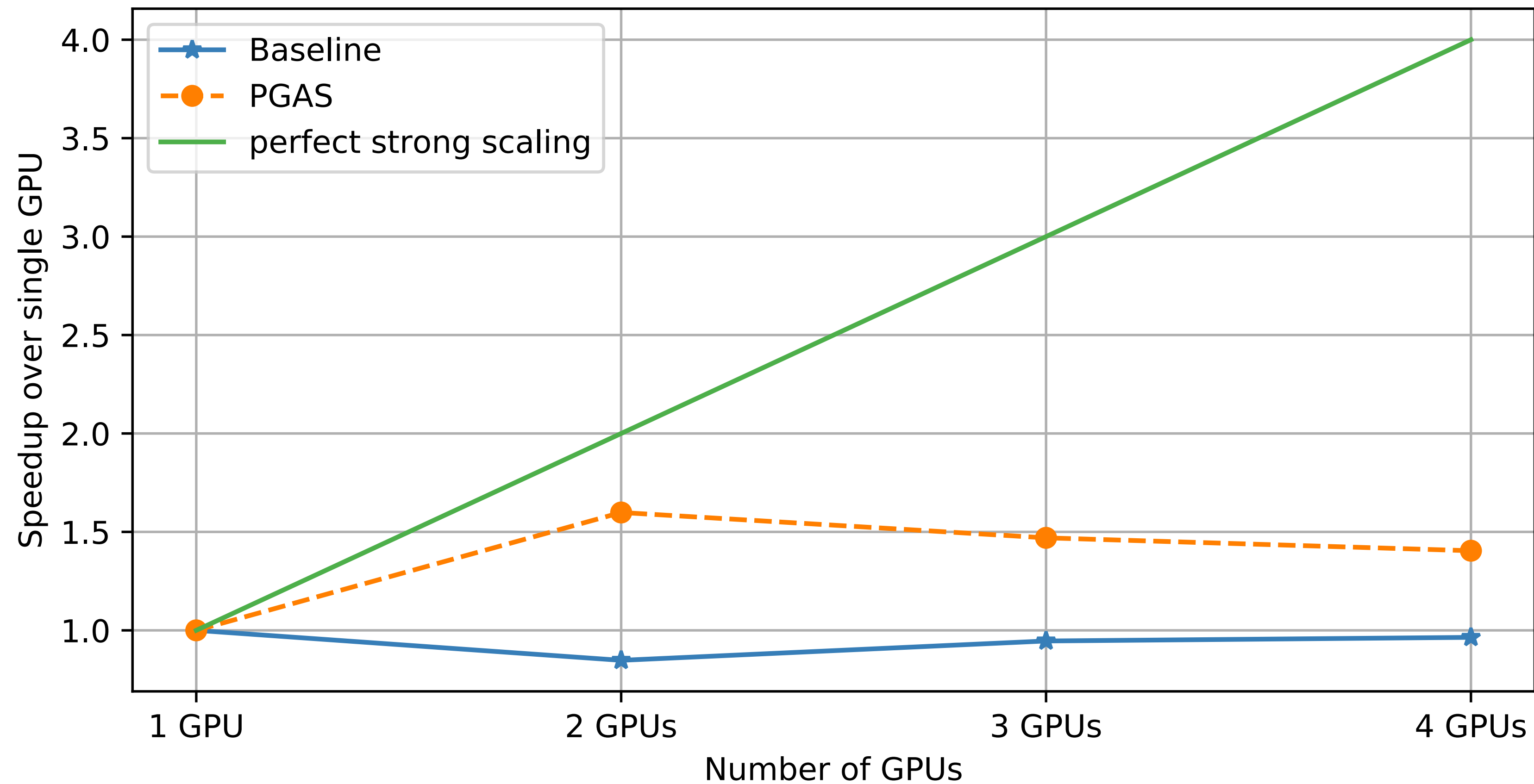
Strong Scaling

- Best overall: **PGAS** implementation
- **PGAS** is able to completely hide the communication with the computation
- **PGAS** reduces sync cost and eliminate the unpack operation

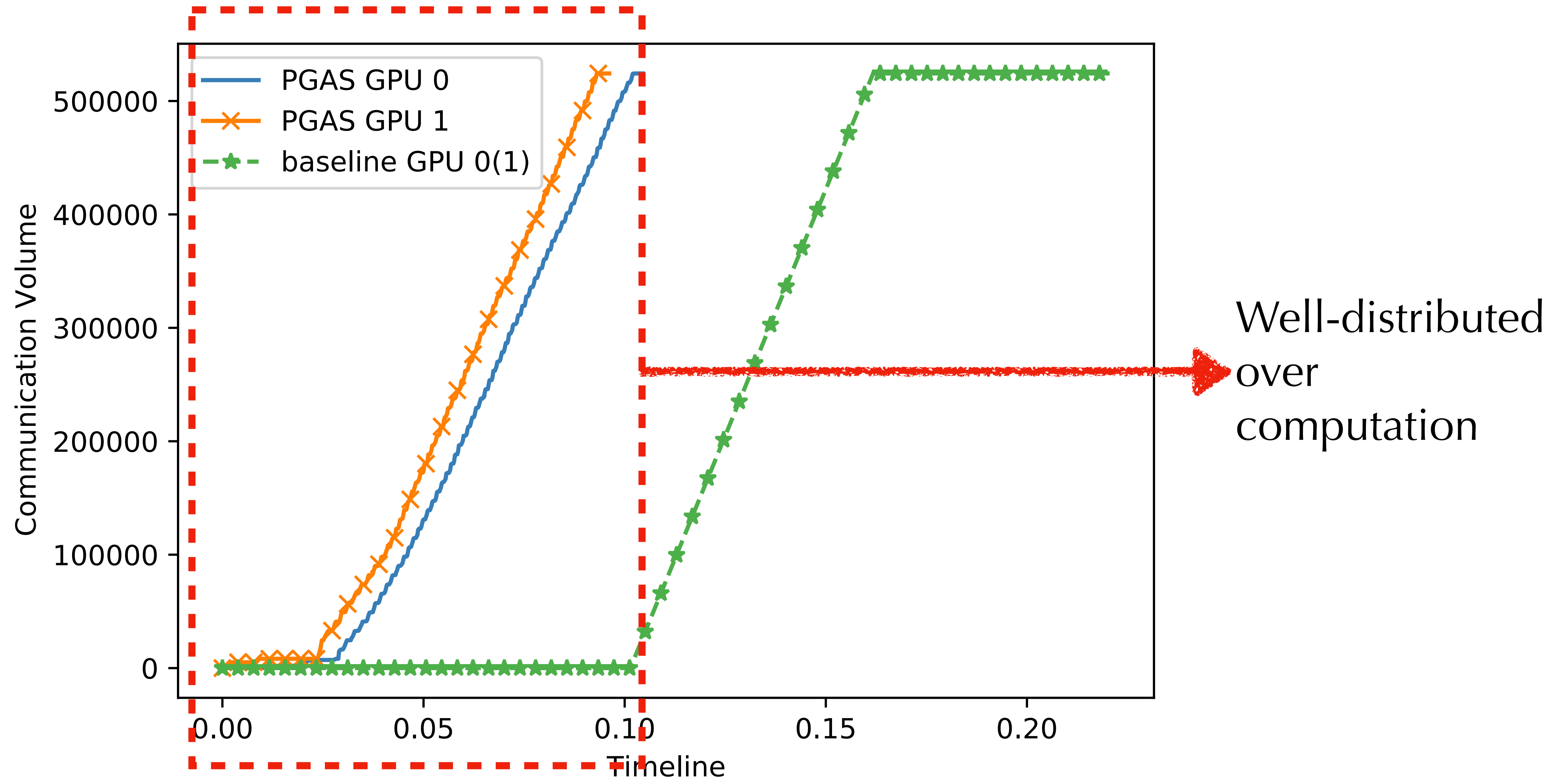


PGAS DLMR Strong Scaling

Strong Scaling



PGAS DLMR Communication Profile



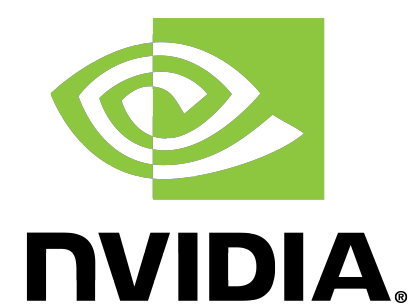
Conclusion

PGAS one-sided async small messages performance benefits:

1. A GPU-centric control path reduces communication latency (issue inside kernel)
2. Fine-grained computation and communication overlap
3. Smooth network usage
4. Eliminating unpack steps

Deep Learning Recommendation System cross-GPU Communication Optimization

Yuxin Chen, Aydin Buluc, Katherine Yelick, John Owens

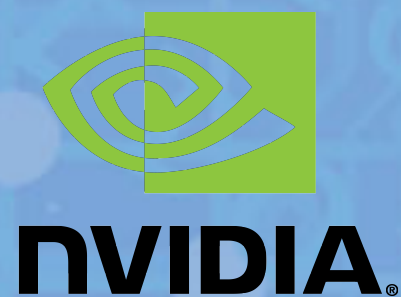


UC DAVIS

Berkeley
UNIVERSITY OF CALIFORNIA



Thank You



UCDAVIS

Berkeley
UNIVERSITY OF CALIFORNIA

Deep Learning Recommendation Model (DLMR) Inferencing

